

CSE 220 Sessional Prep: Section B Predicted Exam Guide

Comprehensive Approaches and Step-by-Step Solutions

Prediction 1 (Task 1: Fourier Series): Shape Transformations

The Probable Question:

“Section A implemented energy-based pruning to compress the shape. For Section B, apply mathematical transformations strictly in the frequency domain. In `fs_redrawer.py`, implement a method to spatially scale and rotate the 2D drawing by a given angle θ , and a method to reverse the drawing direction (time reversal). Print the modified coefficients and save the new visualization.”

The Theoretical Approach:

You do not need to touch the spatial coordinates (x, y) to rotate or scale a shape. Because each coefficient c_n is a complex vector, you can manipulate the entire drawing using basic complex arithmetic properties of the Fourier Series:

- **Scaling & Rotation (Linearity & Phase Shift):** Multiplying every c_n by a complex scalar $Ae^{j\theta}$ scales the radius of every epicycle by A and shifts its starting angle by θ . This seamlessly scales and rotates the final image without recalculating integrals.
- **Time Reversal:** To make the epicycles draw the shape backwards ($t \rightarrow -t$), the Time Reversal property states that $x(-t) \leftrightarrow c_{-n}$. You simply swap the positive frequencies with their negative counterparts.

Step-by-Step Solution:

Step 1: Open `fs_redrawer.py` and locate your `FourierEpicycles` class.

Step 2: Add the following two methods directly into the class. Notice how they operate entirely on the `self.coefs` dictionary in $O(N)$ time.

```
def transform_shape(self, scale_factor, angle_radians):
    """
    Scales the drawing by scale_factor and rotates it by angle_radians.
    """
    # Create the complex transformation vector: A * e^(j * theta)
    transformation_vector = scale_factor * np.exp(1j * angle_radians)

    # Multiply every coefficient by this vector
    for n in self.coefs.keys():
        self.coefs[n] = self.coefs[n] * transformation_vector

def reverse_drawing_direction(self):
    """
    Forces the epicycles to trace the shape backwards using Time Reversal.
    """
    new_coefs = {}
    for n in self.coefs.keys():
        # Map the current coefficient to its negative harmonic slot
        new_coefs[-n] = self.coefs[n]

    # Overwrite the old dictionary
    self.coefs = new_coefs
```

Step 3: In your `__main__` execution block, call these functions after calculating all coefficients, but *before* you call `save_outputs`.

```
# Example execution for main block
fs.calculate_all_coefficients()

# Scale by 2.0 and rotate by 90 degrees (pi/2)
fs.transform_shape(2.0, np.pi/2)

# Reverse the drawing direction
```

```
fs.reverse_drawing_direction()  
  
save_outputs(fs, z, "transformed_comparison.png", "transformed.gif")
```

Prediction 2 (Task 2: CFT): Ideal Low-Pass & Directional Filters

The Probable Question:

“Section A implemented a High-Pass Filter to extract sharp edges. For Section B, implement an Ideal Low-Pass Filter to blur the image, and a Directional Filter to extract *only* vertical or *only* horizontal edges. Apply these filters in `cft_edge_detector.py` and reconstruct the spatial images.”

The Theoretical Approach:

The frequency grid $F(u, v)$ maps spatial changes. The center of the grid (0,0) represents the DC component (average color/smoothness), while the outer edges represent sharp, rapid changes (edges).

- **Low-Pass Filter:** The exact inverse of the high-pass filter. Instead of zeroing out the center, you zero out everything *outside* the cutoff radius. This deletes the sharp edges and leaves only the smooth, blended colors.
- **Directional Filtering:** The u -axis controls horizontal changes, and the v -axis controls vertical changes. If you zero out the u -axis frequencies, the image loses all horizontal texture, leaving only strictly vertical edges.

Step-by-Step Solution:

Step 1: Open `cft_edge_detector.py` and locate the `FrequencyFilter` class.

Step 2: Add the following methods. We avoid slow $O(N^2)$ standard for loops for the low-pass filter and instead use NumPy boolean masking to zero out the arrays instantly.

```
def low_pass(self, real, imag, cutoff):
    """
    Retains only frequencies WITHIN the cutoff radius.
    Result: A blurred/smoothed image with no sharp edges.
    """
    rows, cols = real.shape
    cx, cy = rows // 2, cols // 2

    # Create a meshgrid of coordinates
    y, x = np.ogrid[:rows, :cols]

    # Calculate squared distance from center for every pixel
    dist_squared = (x - cx)**2 + (y - cy)**2

    # Boolean mask: True where distance is GREATER than cutoff (Outside circle)
    mask = dist_squared > cutoff**2

    real_filtered = real.copy()
    imag_filtered = imag.copy()

    # Zero out the high frequencies instantly using the mask
    real_filtered[mask] = 0
    imag_filtered[mask] = 0

    return real_filtered, imag_filtered

def vertical_edge_filter(self, real, imag, u_cutoff):
    """
    Zeros out the horizontal frequencies (u-axis) to isolate vertical edges.
    """
    rows, cols = real.shape
    cx, cy = rows // 2, cols // 2

    real_filtered = real.copy()
    imag_filtered = imag.copy()

    # Iterate and suppress frequencies where the horizontal distance
    # from the center is greater than the u_cutoff
    for i in range(rows):
        for j in range(cols):
            if abs(j - cx) > u_cutoff:
                real_filtered[i, j] = 0
                imag_filtered[i, j] = 0
```

```
return real_filtered, imag_filtered
```